

Engine Management Unit (EMU)

Hardware and Software Reference

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.6

Date: March 2026

This document describes the hardware, software architecture, valve control system, Fill valve handshake protocol, sensor subsystems, buzzer and visual indicator state machine, and control flow of the Engine Management Unit. The EMU is mounted inside the rocket and is the primary actuator and sensor node of the KJO system.

Contents

1	Hardware Overview	2
1.1	Component Summary	2
1.2	MCP23X17 GPIO Expander Pin Map	2
1.3	Hardware Block Diagram	3
2	Software Architecture	3
2.1	Module Summary	3
2.2	Main Loop Control Flow	4
3	Valve Control Subsystem	4
3.1	Valve Base Class (KJO_Valve)	4
3.2	EMU State Machine	4
3.3	Engine Start Sequence (7 Steps)	5
3.4	Servo Mechanical Geometry and Calibration	6
3.5	Master_Valve Class (KJO_Master_Valve)	6
4	Fill Valve Handshake Protocol	6
5	Command Dispatch	7
5.1	Command Table	8
6	Umbilical Quick-Release (QR_Master)	8
6.1	Overview	8
7	Buzzer and Visual Indicator	9
7.1	Startup Self-Test	9
7.2	NeoPixel Indicator State Machine	9
7.3	Buzzer Behaviour	10
7.4	NeoPixel_Indicator API	10
8	Sensor Subsystems	10
8.1	Pressure Transducer Signal Chain	11
8.2	Continuous Pressure Recording	11
9	SD Logging	11
10	Version History	11

1 Hardware Overview

1.1 Component Summary

Component	Part	Interface	Notes
Microcontroller	Adafruit Feather M0 (AT-SAMD21G18A, 48 MHz)	—	
OLED Display	SH1107, 128×64, 1.12"	I ² C	
GPIO Expander	MCP23X17 (16 I/O)	I ² C	Relays, buttons, AUX_IO
PWM Servo Wing	Adafruit 8-channel wing (PCA9685)	I ² C	Ox, Fuel servos
ADC	ADS1015 (12-bit, 4-ch)	I ² C	Ch0: unassigned; Ch1: chamber pressure; Ch2: tank pressure; Ch3: battery voltage
Thermocouple IF	MAX31856 (2×)	SPI	TC1, TC2
Real-Time Clock	PCF8523	I ² C	
LoRa Radio	HopeRF RFM95W (Semtech SX1276), 915 MHz	SPI	
SD Card	Standard SPI SD socket	SPI	
NeoPixel	WS2812B 4×8 array	GPIO	Status indicator
Igniter Relay	SPDT relay via MCP23X17	GPIO	
Buzzer	Passive piezo via MCP23X17 relay	GPIO	
Battery	3.7 V LiPo; voltage divider on ADS1015	ADC	

1.2 MCP23X17 GPIO Expander Pin Map

Pin Name	Direction	Function
BUTTON_A_EIO	IN	Front-panel button A (hold-to-open Ox valve, dumps via chamber)
BUTTON_B_EIO	IN	Front-panel button B (close Fill valve)
BUTTON_C_EIO	IN	Front-panel button C (open Fill valve)
IGNITER_EIO	OUT	Igniter relay drive
BUZZER_EIO	OUT	Buzzer relay drive
AUX_IO_1 (FILL_VALVE_CMD_EIO)	OUT	Fill valve CMD line to GSEMU
AUX_IO_2 (FILL_VALVE_STATUS_EIO)	IN	Fill valve STATUS line from GSEMU
AUX_IO_3 (QR_CMD_EIO)	OUT	Umbilical QR release CMD (HIGH = hold, LOW = release)
AUX_IO_4 (RELEASE_STATE_EIO)	IN	Separation state (INPUT_PULLUP; LOW = connected, HIGH = separated)
AUX_IO_5 (LAUNCH_ENABLE_EIO)	OUT	Remote start enable to GSEMU (HIGH = disabled, LOW = enabled)
AUX_IO_6 (REMOTE_START_EIO)	IN	Remote start command from GSEMU (INPUT_PULLUP; LOW = start)

1.3 Hardware Block Diagram

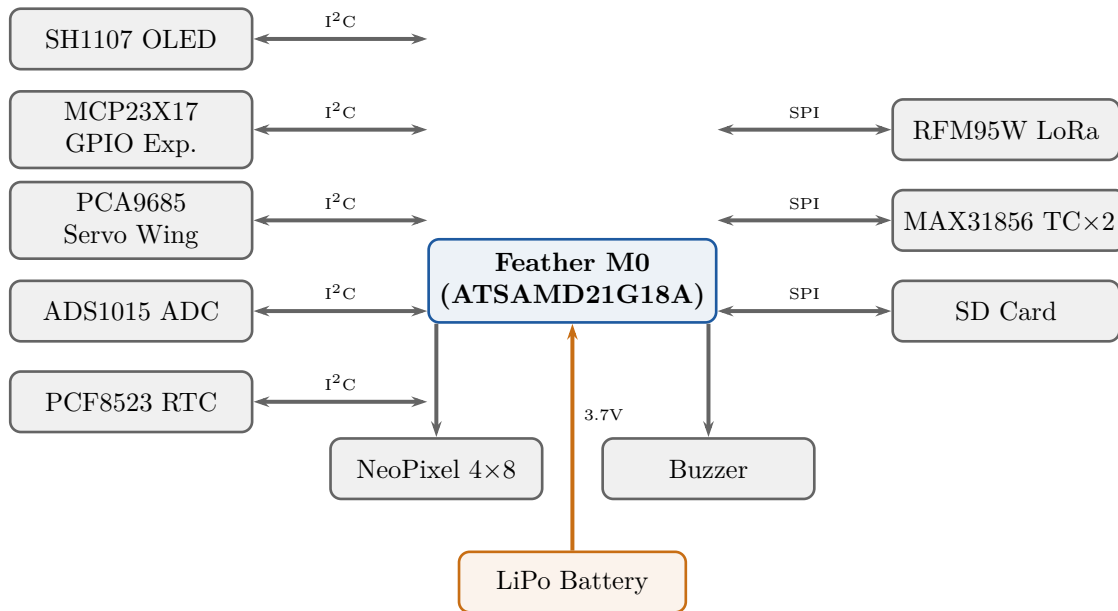


Figure 1: EMU hardware block diagram.

2 Software Architecture

2.1 Module Summary

Module	Location	Responsibility
main.cpp	local	Setup, loop, command dispatch, state machines
KJO_Valve	local	Base valve: servo drive, pot feedback
KJO_Master_Valve	local	Fill valve CMD/STATUS GPIO handshake
KJO_QR_Master	local	Umbilical QR release proxy
KJO_Control_System	local	Execute_Command() routing table
KJO_Pressure	local	Pressure transducer read / convert
KJO_Temperature	local	Analog temperature transducer
KJO_Thermocouple	local	MAX31856 thermocouple read
KJO_Analog	local	ADS1015 ADC wrapper
KJO_GPIO	local	MCP23X17 pin definitions, Relay class
KJO_EMU_Display	local	SH1107 OLED display functions
KJO_Indicator	local	NeoPixel status indicator (solid, timed, flash)
KJO_Logging	local	SD log file constants
KJO_Radio	shared	Radio pin constants, frequency
KJO_Command_Defs	shared	Command codes, packet structures
KJO_System_Config	shared	Version, platform constants

2.2 Main Loop Control Flow

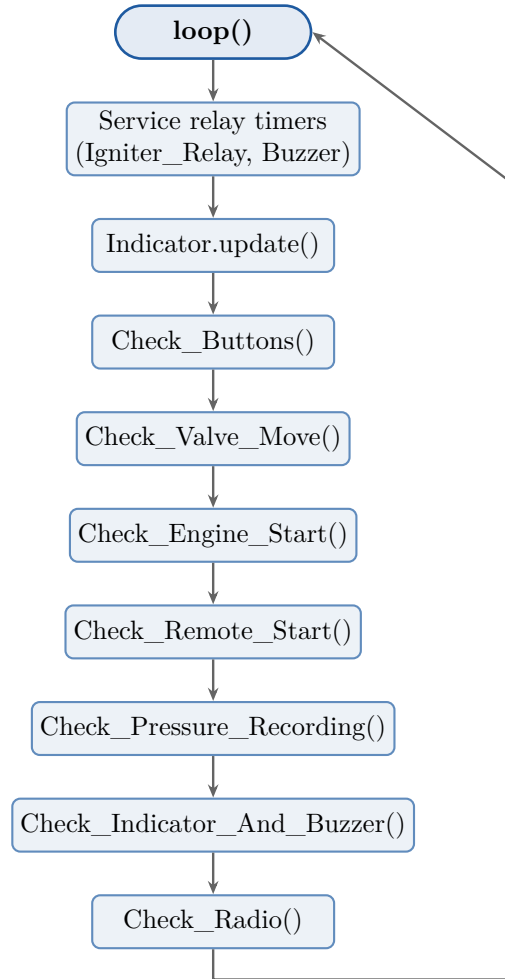


Figure 2: EMU main loop control flow.

3 Valve Control Subsystem

3.1 Valve Base Class (KJO_Valve)

The **Valve** class encapsulates a servo-driven, potentiometer-feedback valve:

- **Drive:** PWM signal to PCA9685 servo wing channel.
- **Feedback:** Potentiometer on ADS1015 ADC channel; position expressed as 0–100% open.
- **Position control:** **openTo(%)** sets target; **update()** drives servo until pot reading reaches target within dead-band.
- **State queries:** **isMoving()**, **isStopped()**, **isOpen()**, **isClosed()**.

3.2 EMU State Machine

The EMU operates as a top-level state machine with three states:

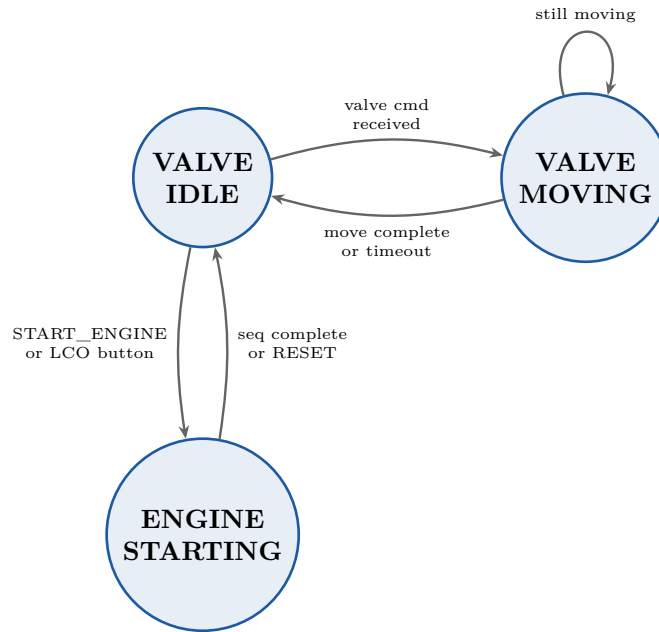


Figure 3: EMU top-level state machine. HEARTBEAT and RESET are processed in any state. All other commands are dropped during VALVE_MOVING or ENGINE_STARTING.

3.3 Engine Start Sequence (7 Steps)

When **START_ENGINE** or a LCO panel button press is received, the EMU enters **ENGINE_STARTING** and advances through seven steps non-blockingly in **Check_Engine_Start()**:

Step	Trigger	Action
0	immediate	Verify Ox and Fuel valves are closed; abort on failure
1	step 0 pass	Command umbilical QR release; start 3 s separation timer
2	timer	Poll isSeparated() ; abort on timeout
3	separation	Fire igniter relay (IGNITER_FIRE_DURATION_MS pulse); start delay-1 timer
4	delay-1	Open Ox feed valve
5	Ox stopped	Start delay-2 timer
6	delay-2	Open Fuel feed valve
7	Fuel stopped	Send OK response; set engine_running = true

Any abort at steps 0–2 sends a **Command_Return_Packet** with **status = false** and an **r_val** error code (–1 = precondition failure, –2 = QR timeout). **RESET** cancels the sequence at any step.

3.4 Servo Mechanical Geometry and Calibration

Gear Train and Servo Travel

Each valve is actuated by a ServoCity 2000-0025-0504 5-turn servo through a 16-tooth pinion driving a 60-tooth valve gear:

Parameter	Value
Gear ratio	$60 \div 16 = \mathbf{3.75 : 1}$
Valve travel (open $\rightarrow$ close)	90°
Required servo rotation	$90 \times 3.75 = 337.5^\circ$
Servo full mechanical range	1800° (5 turns) over 500–2500 μs
PCA9685 frequency	1100 Hz (non-standard; improves count resolution)
PWM period at 1100 Hz	$1 \div 1100 = 909.09 \mu\text{s}$
μs per 12-bit count	$909.09 \div 4096 \approx 0.2219 \mu\text{s}/\text{count}$
μs for 337.5° servo travel	$337.5 \times (2000 \mu\text{s} \div 1800^\circ) = 375 \mu\text{s}$
Theoretical Δ counts	$375 \div 0.2219 \approx \mathbf{1690}$ counts

Note

Running the PCA9685 at 1100 Hz (instead of the standard 50 Hz) compresses the usable pulse-width span into the lower portion of the 12-bit count range, effectively improving resolution over the ~ 1.1 turns of servo travel actually used.

Empirical Calibration (March 2026)

Bench testing showed the theoretical 1690-count span caused all four valves to travel noticeably *more* than 90° . All three valves were re-calibrated. The empirical open-to-close deltas average ≈ 1610 counts ($\sim 5\%$ below theoretical), consistent with the servo being slightly more sensitive in the operating band than the linear model predicts.

Valve	Servo PWM (12-bit counts)				Pot ADC (12-bit counts)		
	Open	Close	Δ	Implied travel	Open	Closed	Δ
Fill	2179	3719	1540	$\sim 82.1^\circ$	1238	2984	1746
Oxidizer	2050	3690	1640	$\sim 87.4^\circ$	1139	2857	1718
Fuel	2130	3780	1650	$\sim 87.9^\circ$	1170	2918	1748

3.5 Master_Valve Class (KJO_Master_Valve)

`Master_Valve` implements the CMD/STATUS GPIO handshake for the Fill valve. The Fill valve servo and pot reside on the GSEMU; the EMU controls it via `AUX_IO_1` (CMD) and monitors `AUX_IO_2` (STATUS).

4 Fill Valve Handshake Protocol

The Fill valve servo and potentiometer reside on the GSEMU. The EMU and GSEMU exchange state using two GPIO lines:

Signal	Driver	Meaning when HIGH
<code>AUX_IO_1</code> (CMD)	EMU	Fill valve commanded open
<code>AUX_IO_2</code> (STATUS)	GSEMU	Fill valve has reached commanded position

Open sequence:

1. EMU calls `Fill_Valve.open()`.

2. `Master_Valve` asserts `CMD = HIGH`.
3. `GSEMU Slave_Valve.update()` detects `CMD` rising edge; drives servo to open.
4. `GSEMU` asserts `STATUS = HIGH` when servo reaches open position.
5. `EMU` reads `STATUS = HIGH`; reports completion.

Close sequence: `CMD` de-asserted (`LOW`); `GSEMU` closes servo; `STATUS` driven `LOW`.

5 Command Dispatch

`Check_Radio()` receives packets from the RCU and routes them. Commands are processed in priority order:

1. **HEARTBEAT** — always processed; returns `Heartbeat_Response_Packet` with `EMU` voltage and all three valve positions.
2. **RESET** — always processed regardless of state; calls `Execute_Abort()`.
3. All other commands are *dropped* if `emu_state` is `VALVE_MOVING` or `ENGINE_STARTING`.
4. Per-command interceptors (deferred valve moves, start sequence, recording).
5. All remaining commands dispatched to `Execute_Command()`.

5.1 Command Table

Command	EMU Action
HEARTBEAT	Respond with EMU voltage and all three valve positions
RESET	Execute_Abort(): Fill/Fuel close, Ox opens (dump), igniter off, buzzer off, recording off, indicator off
BEGIN_FILL	Open Fill valve (Master_Valve); start continuous pressure recording
STOP_FILL	Close Fill valve; recording continues until ambient
DUMP	Open Ox valve to vent pressurant through the chamber (deferred; sends response when complete; no dedicated Dump valve)
STOP_DUMP	Close Ox valve (deferred)
OPEN_OXIDIZER_FEED	Open Ox valve (deferred)
CLOSE_OXIDIZER_FEED	Close Ox valve (deferred)
OPEN_FUEL_FEED	Open Fuel valve (deferred)
CLOSE_FUEL_FEED	Close Fuel valve (deferred)
START_ENGINE_FLOWS	Close Fill; open Ox + Fuel simultaneously (deferred dual-valve)
STOP_ENGINE_FLOWS	Close Ox + Fuel simultaneously (deferred dual-valve)
START_ENGINE	Arm 7-step ignition sequence (response sent asynchronously)
ENABLE_LCO_CONTROL	param $\neq$ 0: assert LAUNCH_ENABLE_EIO LOW, set <code>lco_enabled</code> ; param = 0: deassert HIGH, clear <code>lco_enabled</code>
START_PRESSURE_RECORDING	Activate continuous sensor recording
STOP_PRESSURE_RECORDING	Deactivate continuous sensor recording
READ_VALVE_POSITION	Return current position (%) of the valve specified in <code>param</code>
READ_TANK_PRESSURE	Return tank pressure (PSI)
READ_CHAMBER_PRESSURE	Return chamber pressure (PSI)
READ_TANK_TEMP	Return tank temperature
READ_CHAMBER_TEMP	Return chamber temperature
QR_RELEASE	Call <code>QR_Master::release()</code> (CMD LOW); response immediate

6 Umbilical Quick-Release (QR_Master)

6.1 Overview

QR_Master is the EMU-side proxy for the umbilical quick-release actuator that resides on the GSEMU. It drives two MCP23X17 expansion I/O lines:

- AUX_IO_3 (QR_CMD_EIO) — OUTPUT. Level-based command to the GSEMU QR servo.
- AUX_IO_4 (RELEASE_STATE_EIO) — INPUT_PULLUP. Wired to GSEMU chassis GND

through the AUX connector. Reads LOW while the umbilical is physically connected; floats HIGH when the connector is removed (separation confirmed).

QR_CMD_EIO Level	Driver	Meaning
HIGH (default)	EMU	Latch engaged — GSEMU servo held at hold position
LOW	EMU	Latch open — GSEMU servo commanded to release position

Warning

Ensure the launch sequence has been verified and all personnel are clear before issuing **QR_RELEASE**. The umbilical disconnection releases pressurised N₂O. Calling `hold()` after `release()` returns the CMD line HIGH but cannot re-engage a latch that has already opened mechanically.

7 Buzzer and Visual Indicator

7.1 Startup Self-Test

At power-on, the buzzer fires **five short beeps** (75 ms on / 75 ms gap) to confirm the buzzer and relay circuits are functional. The NeoPixel cycles red → yellow → green → off as a pixel self-test.

7.2 NeoPixel Indicator State Machine

`Check_Indicator_And_Buzzer()` runs every loop iteration and updates the 32-pixel NeoPixel array to reflect the highest-priority active condition. States are evaluated in descending priority:

Priority	Condition	Pattern	Colour
1	<code>engine_running = true</code>	Off (all pixels dark)	—
2	<code>emu_state = ENGINE_STARTING</code>	Solid	Red
3	<code>tank_pressurized = true</code>	Flash 300/500 ms	Red
4	<code>link_connected = true</code>	Flash 300/1200 ms	Green
5	(unlinked)	Flash 300/1200 ms	Yellow

`link_connected` is set true on every received **HEARTBEAT** and cleared after 15 s of silence (loss-of-signal watchdog). `tank_pressurized` is updated by `Check_Pressure_Recording()` using the live tank pressure reading; the threshold is `> PRESSURE_WARNING_THRESHOLD` (15 PSI).

7.3 Buzzer Behaviour

Condition	Buzzer Action
Startup self-test	5 beeps (75 ms on, 75 ms gap) — blocking, in <code>setup()</code>
Button press confirmation	1 or 2 short beeps (100 ms) via <code>Beep()</code>
Pressure alarm	Continuous tone while <code>tank_pressurized = true</code>
Engine start / abort	Alarm silenced immediately when <code>Arm_Engine_Start()</code> or <code>Execute_Abort()</code> is called
“I’m alive” heartbeat	Single 100 ms beep every 10 s; suppressed during alarm, <code>ENGINE_STARTING</code> , and <code>ENGINE_RUNNING</code>

The pressure alarm is managed entirely within `Check_Indicator_And_Buzzer()`. When the indicator state transitions *to* `IND_PRESSURIZED`, `Buzzer.on()` is called (no timeout — continuous). When the state transitions *away* from `IND_PRESSURIZED`, `Buzzer.off()` is called. `Arm_Engine_Start()` and `Execute_Abort()` both call `Buzzer.off()` directly and clear the `pressure_alarm_active` flag so the next `Check_Indicator_And_Buzzer()` call does not re-arm it.

7.4 NeoPixel_Indicator API

Method	Action
<code>begin()</code>	Initialise hardware; clear all pixels
<code>on(color)</code>	Set all pixels solid; cancel any flash or timed mode
<code>onRed()</code> / <code>onGreen()</code> / <code>onYellow()</code>	Convenience wrappers
<code>flash(color, on_ms, off_ms)</code>	Periodic flash indefinitely; cancelled by <code>on()</code> or <code>off()</code>
<code>off()</code> / <code>clear()</code>	All pixels off; cancel flash/timed modes
<code>update()</code>	Service timed shutoff and flash state machine — call from <code>loop()</code>

8 Sensor Subsystems

Sensor	Class	ADC / Bus	Measurement
Chamber pressure transducer	Pressure_Transducer	ADS1015 ch1	Combustion chamber pressure, 0–200 PSI
Tank pressure transducer	Pressure_Transducer	ADS1015 ch2	N ₂ O tank pressure, 0–1000 PSI
Battery voltage	—	ADS1015 ch3	LiPo pack voltage (via resistor divider)
Tank temperature sensor	Temperature_Transducer	Onboard ADC (A0)	N ₂ O tank temperature [†]
Chamber temperature sensor	Temperature_Transducer	Onboard ADC (A5)	Chamber wall temperature [†]
Thermocouple 1	MAX31856	SPI	TC1 (location TBD)
Thermocouple 2	MAX31856	SPI	TC2 (location TBD)

[†]Onboard ADC channel assignments are placeholders pending hardware schematic confirmation.

8.1 Pressure Transducer Signal Chain

Each pressure transducer produces a ratiometric voltage output over 0.5–4.5 V corresponding to 0–full-scale PSI. A voltage divider on the A/D daughter board scales this to the ADS1015 input range:

$$V_{\text{ADC}} = V_{\text{transducer}} \times \frac{3.3}{4.5}$$

Condition	Transducer (V)	After divider (V)	ADS1015 count (GAIN_ONE)
0 PSI (zero offset)	0.500	0.367	≈ 184
Full-scale PSI	4.500	3.300	≈ 1657

ADS1015 at GAIN_ONE (± 4.096 V): ≈ 0.001991 V/count. The two-point linear `map()` in `Pressure_Sensor_Average()` implicitly performs all three conversion steps (undo divider, subtract 0.5 V offset, scale 0–4 V to PSI) without explicit voltage arithmetic.

Constant	Value	Meaning
PRESSURE_MIN_READING	184	ADS1015 counts at 0 PSI (0.500 V transducer)
PRESSURE_MAX_READING	1657	ADS1015 counts at full-scale PSI (4.500 V transducer)
CHAMBER_MAX_PSI	200	Chamber transducer full-scale (PSI)
TANK_MAX_PSI	1000	Tank transducer full-scale (PSI)

8.2 Continuous Pressure Recording

Continuous recording is started automatically by `BEGIN_FILL` and may also be started manually with `START_PRESSURE_RECORDING`. Every `PRESSURE_RECORDING_INTERVAL_MS` (250 ms) the EMU reads tank pressure, chamber pressure, tank temperature, and chamber temperature and appends all four values to the SD log.

Recording auto-stops when the tank pressure remains below `PRESSURE_AMBIENT_THRESHOLD` (10 PSI) for four consecutive readings, or when `STOP_PRESSURE_RECORDING` or `RESET` is received.

9 SD Logging

Log files are stored in the root of the SD card as `Log_N.txt` (N is auto-incremented to avoid overwriting existing files). Each record is time-stamped (PCF8523) and records command receipt, valve movements, sensor readings, and fault events.

10 Version History

Version	Date	Notes
0.1	March 2026	Initial release.
0.2	March 2026	Add umbilical QR release subsystem (<code>QR_Master</code> , <code>QR_RELEASE</code> command, <code>AUX_IO_3</code> protocol).
0.3	March 2026	Level-based QR protocol (release=LOW, hold=HIGH); add <code>RELEASE_STATE_EIO</code> , <code>LAUNCH_ENABLE_EIO</code> , <code>REMOTE_START_EIO</code> ; Button A reassigned to hold-to-open Dump valve; <code>QR_Master</code> API updated.
0.4	March 2026	Full command dispatch table; <code>ENGINE_STARTING</code> state added to state machine; 7-step engine start sequence documented; new §7 Buzzer and Visual Indicator: 5-beep startup, NeoPixel state machine (5 states/priorities), continuous pressure-alarm buzzer, 10 s alive beep, alarm silence on start/abort.
0.5	March 2026	Valve servo calibration: added §3 servo geometry subsection (gear ratio 3.75:1, 1100 Hz PCA9685, ≈ 1690 theoretical counts for 90°) and empirical per-valve calibration table (all four valves re-calibrated on bench). Pressure transducer fix: corrected ADS1015 channel assignments (chamber = ch1/200 PSI, tank = ch2/1000 PSI), corrected sensor table (temperature sensors use onboard ADC A0/A5, not ADS1015), added §7 signal chain subsection documenting voltage divider and calibration endpoint counts (184/1657).
0.6	March 2026	SD log files moved to SD card root (<code>Log_N.txt</code>); subdirectory path constants (<code>LOG_FILE_FOLDER</code> , <code>LOG_FILE_DIR</code>) removed from <code>KJO_Logging.h</code> and <code>KJO_EMU_Display</code> .
0.7	July 2026	Dump valve physically removed from the vehicle; pressurant is now dumped through the combustion chamber via the Ox valve. <code>DUMP/STOP_DUMP</code> retained but now target the Ox valve; <code>Execute_Abort()</code> closes Fill/Fuel and opens Ox (instead of closing all valves); Button A reassigned to hold-to-open the Ox valve; <code>dump_pct</code> removed from <code>Heartbeat_Response_Packet</code> .